

JAVA

60 exercices d'application — Thèmes 1 à 6

10 exercices par thème, à faire par toi-même. Les corrections complètes se trouvent à la toute fin du document.

Thème 1 — Syntaxe de base

Thème 2 — Programmation Orientée Objet

Thème 3 — Collections & génériques

Thème 4 — Streams & lambdas

Thème 5 — Design Patterns

Thème 6 — JUnit 5 & JDBC

THÈME 1 — Syntaxe de base

Exercices

Exercice 1

Déclare une variable de chaque type primitif (int, double, boolean, char) avec des valeurs cohérentes, puis affiche-les toutes.

Exercice 2

Écris un programme qui, à partir d'une variable int age, affiche "Majeur" ou "Mineur" avec if/else.

Exercice 3

Utilise une boucle for pour afficher tous les nombres pairs de 0 à 20 (inclus).

Exercice 4

Utilise une boucle while pour calculer la somme des entiers de 1 à 100.

Exercice 5

Écris un switch moderne (arrow, Java 14+) qui affiche le nom du mois à partir d'un numéro (1 à 12).

Exercice 6

Écris une méthode statique multiplier(int a, int b) qui retourne le produit, puis appelle-la dans un main.

Exercice 7

Montre par un exemple la différence entre int et Integer concernant la valeur null.

Exercice 8

Affiche un triangle d'étoiles de hauteur 5 avec des boucles for imbriquées.

Exercice 9

Écris une méthode boolean estPair(int n) et teste-la avec plusieurs valeurs.

Exercice 10

Parcours un tableau d'entiers avec une boucle for et affiche "pair" ou "impair" pour chaque élément.

THÈME 2 — Programmation Orientée Objet

Exercices

Exercice 1

Crée une classe Voiture avec attributs marque et vitesse, un constructeur, et une méthode accélérer() qui augmente la vitesse de 10.

Exercice 2

Crée une classe Animal avec une méthode faireBruit(), puis Chien et Chat qui l'étendent et la redéfinissent avec @Override.

Exercice 3

Crée une interface Volant avec une méthode voler(), puis fais-la implémenter par Oiseau et Avion.

Exercice 4

Crée une classe abstraite Forme avec une méthode abstraite aire(), et implémente-la pour Rectangle et Triangle.

Exercice 5

Démontre le polymorphisme avec une List contenant des Chien et des Chat, en appelant faireBruit() sur chacun.

Exercice 6

Utilise this pour distinguer un attribut d'un paramètre dans le constructeur d'une classe Livre(titre, auteur).

Exercice 7

Utilise super pour appeler le constructeur parent dans une classe Manager qui hérite d'Employe.

Exercice 8

Crée une classe CompteBancaire avec un attribut privé solde, un getter, et un setter qui empêche un solde négatif.

Exercice 9

Crée une classe Triathlete qui implémente deux interfaces Nageur et Coureur.

Exercice 10

Illustre par du code la différence entre classe abstraite et interface, pour un même besoin (ex: Vehicule).

THÈME 3 — Collections & génériques

Exercices

Exercice 1

Crée une ArrayList de 5 prénoms, affiche-la, puis supprime l'élément à l'index 2 et affiche le résultat.

Exercice 2

Ajoute les entiers 1, 2, 2, 3, 3, 3 dans un HashSet et affiche sa taille. Compare avec une List.

Exercice 3

Crée un TreeSet avec des prénoms ajoutés dans le désordre. Affiche le résultat et observe le tri automatique.

Exercice 4

Crée une HashMap associant 4 prénoms à des âges, puis utilise getOrDefault pour une clé absente.

Exercice 5

Parcours une Map avec entrySet() et affiche chaque paire sous la forme "Nom : Age".

Exercice 6

Écris une méthode générique T max(List liste, Comparator comp) qui retourne le plus grand élément.

Exercice 7

Crée une classe générique Boite avec un attribut contenu, un getter et un setter. Teste avec String puis Integer.

Exercice 8

Trie une List d'abord par nom, puis par âge en cas d'égalité de nom, avec Comparator.comparing/thenComparing.

Exercice 9

Insère les mêmes 5 paires clé-valeur dans un HashMap et un LinkedHashMap, affiche les deux et compare l'ordre.

Exercice 10

À partir d'une List, construis à la main (sans Streams) une Map<> groupée par tranche d'âge ("mineur"/"majeur").

THÈME 4 — Streams & lambdas

Exercices

Exercice 1

Écris une lambda Comparator qui trie une liste d'entiers par ordre décroissant.

Exercice 2

Crée une interface fonctionnelle Operation avec une méthode appliquer(int, int), puis implémente une soustraction en lambda.

Exercice 3

À partir d'une List, garde les mots commençant par une voyelle et mets-les en majuscules avec un Stream.

Exercice 4

Calcule la somme des âges d'une liste de Personne avec mapToInt et sum().

Exercice 5

Utilise reduce() pour concaténer une liste de mots en une seule String séparée par des tirets.

Exercice 6

Compte le nombre de personnes majeures dans une liste avec count().

Exercice 7

Trie une List par âge décroissant avec Comparator.comparing(...).reversed() dans un Stream.

Exercice 8

Regroupe une liste de mots par leur longueur avec Collectors.groupingBy().

Exercice 9

Calcule l'âge moyen des personnes majeures d'une liste avec un Stream (filter + mapToInt + average).

Exercice 10

Écris une méthode filtrerPersonnes(List, Predicate) qui retourne les personnes validant une condition donnée.

THÈME 5 — Design Patterns

Exercices

Exercice 1

Implémente un Singleton pour une classe Logger (une seule instance, méthode log(String message)).

Exercice 2

Implémente une Factory qui crée des objets Vehicule (Voiture, Moto) selon un type String donné.

Exercice 3

Implémente un Builder pour une classe Burger (pain, viande, fromage) avec des valeurs par défaut.

Exercice 4

Implémente le pattern Observer : une MeteoStation notifie des Afficheurs quand la température change.

Exercice 5

Implémente une Strategy pour trier une liste selon deux stratégies : TrierCroissant et TrierDecroissant.

Exercice 6

Réécrit le Singleton Logger en utilisant un enum, et explique pourquoi c'est thread-safe par nature.

Exercice 7

Combine Factory + Strategy : une Factory retourne la bonne StrategiePaiement selon un type donné.

Exercice 8

Ajoute un deuxième Afficheur au pattern Observer précédent et montre que les deux reçoivent la notification.

Exercice 9

Utilise le Builder de la classe Pizza pour créer deux pizzas différentes en changeant seulement certains paramètres.

Exercice 10

Identifie et code le pattern adapté pour garantir une connexion unique à une base de données dans une application.

THÈME 6 — JUnit 5 & JDBC

Exercices

Exercice 1

Écris un test JUnit 5 qui vérifie que `Calcul.diviser(10, 2)` retourne 5.

Exercice 2

Écris un test qui vérifie qu'une `ArithmeticException` est levée lors d'une division par zéro.

Exercice 3

Utilise `@BeforeEach` pour initialiser un objet `Calcul` avant chaque test de la classe.

Exercice 4

Écris 3 tests (`assertEquals`, `assertTrue`, `assertFalse`) pour une méthode `estPositif(int n)`.

Exercice 5

Utilise `@BeforeAll` pour afficher un message une seule fois avant tous les tests de la classe.

Exercice 6

Écris le squelette JDBC d'une requête qui récupère toutes les personnes de plus de 18 ans depuis la table `personnes`.

Exercice 7

Écris une requête JDBC d'insertion avec `PreparedStatement` pour ajouter une personne (nom, age).

Exercice 8

Explique et illustre pourquoi `PreparedStatement` est préférable à `Statement` pour une requête paramétrée.

Exercice 9

Écris une méthode DAO `getPersonneParNom(String nom)` qui récupère une personne unique via JDBC.

Exercice 10

Explique comment tu testerais une méthode DAO sans te connecter à une vraie base, en mentionnant Mockito.

CORRECTIONS

Voici une proposition de solution pour chaque exercice. D'autres approches correctes sont possibles.

Thème 1 — Syntaxe de base

Exercice 1 — Correction

```
int age = 20;
double moyenne = 14.5;
boolean estInscrite = true;
char initiale = 'A';

System.out.println(age);
System.out.println(moyenne);
System.out.println(estInscrite);
System.out.println(initiale);
```

Exercice 2 — Correction

```
int age = 21;
if (age >= 18) {
    System.out.println("Majeur");
} else {
    System.out.println("Mineur");
}
```

Exercice 3 — Correction

```
for (int i = 0; i <= 20; i++) {
    if (i % 2 == 0) {
        System.out.println(i);
    }
}
```

Exercice 4 — Correction

```
int somme = 0;
int i = 1;
while (i <= 100) {
    somme += i;
    i++;
}
System.out.println(somme); // 5050
```

Exercice 5 — Correction

```
int numeroMois = 3;
String mois = switch (numeroMois) {
    case 1 -> "Janvier";
    case 2 -> "Février";
    case 3 -> "Mars";
    case 4 -> "Avril";
    default -> "Autre mois";
};
System.out.println(mois);
```

Exercice 6 — Correction

```
public static int multiplier(int a, int b) {
    return a * b;
}

public static void main(String[] args) {
```



```
        System.out.println(multiplier(4, 5)); // 20
    }
```

Exercice 7 — Correction

```
int a = 5;
// int a = null; // ERREUR de compilation, impossible

Integer b = 5;
Integer c = null; // autorisé car Integer est un objet
System.out.println(c); // affiche "null" sans erreur
```

Exercice 8 — Correction

```
for (int i = 1; i <= 5; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Exercice 9 — Correction

```
public static boolean estPair(int n) {
    return n % 2 == 0;
}

public static void main(String[] args) {
    System.out.println(estPair(4)); // true
    System.out.println(estPair(7)); // false
}
```

Exercice 10 — Correction

```
int[] nombres = {3, 8, 15, 20, 7};
for (int n : nombres) {
    if (n % 2 == 0) {
        System.out.println(n + " est pair");
    } else {
        System.out.println(n + " est impair");
    }
}
```

Thème 2 — Programmation Orientée Objet

Exercice 1 — Correction

```
public class Voiture {
    private String marque;
    private int vitesse;

    public Voiture(String marque) {
        this.marque = marque;
        this.vitesse = 0;
    }

    public void accelerer() {
        this.vitesse += 10;
    }

    public void afficher() {
        System.out.println(marque + " roule à " + vitesse + " km/h");
    }
}
```

Exercice 2 — Correction

```
public class Animal {
    public void faireBruit() {
        System.out.println("...");
    }
}

public class Chien extends Animal {
    @Override
    public void faireBruit() {
        System.out.println("Wouf !");
    }
}

public class Chat extends Animal {
    @Override
    public void faireBruit() {
        System.out.println("Miaou !");
    }
}
```

Exercice 3 — Correction

```
public interface Volant {
    void voler();
}

public class Oiseau implements Volant {
    @Override
    public void voler() { System.out.println("L'oiseau bat des ailes"); }
}

public class Avion implements Volant {
    @Override
    public void voler() { System.out.println("L'avion utilise ses réacteurs"); }
}
```

Exercice 4 — Correction

```
public abstract class Forme {
    public abstract double aire();
}

public class Rectangle extends Forme {
    private double largeur, hauteur;
```

```

    public Rectangle(double l, double h) { largeur = l; hauteur = h; }
    @Override
    public double aire() { return largeur * hauteur; }
}

public class Triangle extends Forme {
    private double base, hauteur;
    public Triangle(double b, double h) { base = b; hauteur = h; }
    @Override
    public double aire() { return (base * hauteur) / 2; }
}

```

Exercice 5 — Correction

```

List<Animal> animaux = new ArrayList<>();
animaux.add(new Chien());
animaux.add(new Chat());

for (Animal a : animaux) {
    a.faireBruit(); // exécute la version réelle de chaque objet
}

```

Exercice 6 — Correction

```

public class Livre {
    private String titre;
    private String auteur;

    public Livre(String titre, String auteur) {
        this.titre = titre; // this.titre = attribut, titre = paramètre
        this.auteur = auteur;
    }
}

```

Exercice 7 — Correction

```

public class Employe {
    protected String nom;
    public Employe(String nom) { this.nom = nom; }
}

public class Manager extends Employe {
    private int nbSubordonnes;
    public Manager(String nom, int nbSubordonnes) {
        super(nom); // appelle le constructeur d'Employe
        this.nbSubordonnes = nbSubordonnes;
    }
}

```

Exercice 8 — Correction

```

public class CompteBancaire {
    private double solde;

    public double getSolde() { return solde; }

    public void setSolde(double solde) {
        if (solde < 0) {
            throw new IllegalArgumentException("Le solde ne peut pas être négatif");
        }
        this.solde = solde;
    }
}

```

Exercice 9 — Correction

```

public interface Nageur { void nager(); }
public interface Coureur { void courir(); }

```

```
public class Triathlete implements Nageur, Coureur {
    @Override
    public void nager() { System.out.println("Je nage"); }
    @Override
    public void courir() { System.out.println("Je cours"); }
}
```

Exercice 10 — Correction

```
// Version classe abstraite : état + logique partagée
public abstract class VehiculeAbstrait {
    protected int vitesseMax;
    public VehiculeAbstrait(int v) { this.vitesseMax = v; }
    public abstract void demarrer();
}

// Version interface : simple contrat, pas d'état
public interface VehiculeInterface {
    void demarrer();
}

// Une classe peut implémenter plusieurs interfaces,
// mais hériter d'une seule classe abstraite.
```

Thème 3 — Collections & génériques

Exercice 1 — Correction

```
List<String> prenom = new ArrayList<>();
prenom.add("Ainhoa");
prenom.add("Marie");
prenom.add("Tom");
prenom.add("Alice");
prenom.add("Léo");

System.out.println(prenom);
prenom.remove(2); // supprime "Tom"
System.out.println(prenom);
```

Exercice 2 — Correction

```
Set<Integer> set = new HashSet<>();
set.add(1); set.add(2); set.add(2);
set.add(3); set.add(3); set.add(3);
System.out.println(set.size()); // 3, doublons ignorés

List<Integer> liste = new ArrayList<>(List.of(1, 2, 2, 3, 3, 3));
System.out.println(liste.size()); // 6, doublons conservés
```

Exercice 3 — Correction

```
Set<String> prenom = new TreeSet<>();
prenom.add("Marie");
prenom.add("Ainhoa");
prenom.add("Tom");
prenom.add("Alice");

System.out.println(prenom);
// [Ainhoa, Alice, Marie, Tom] -> trié alphabétiquement automatiquement
```

Exercice 4 — Correction

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Ainhoa", 21);
ages.put("Marie", 30);
ages.put("Tom", 17);
ages.put("Alice", 25);

System.out.println(ages.get("Ainhoa")); // 21
System.out.println(ages.getOrDefault("Léo", 0)); // 0, clé absente
```

Exercice 5 — Correction

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Ainhoa", 21);
ages.put("Marie", 30);

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}
```

Exercice 6 — Correction

```
public static <T> T max(List<T> liste, Comparator<T> comp) {
    T maxCourant = liste.get(0);
    for (T element : liste) {
        if (comp.compare(element, maxCourant) > 0) {
            maxCourant = element;
        }
    }
    return maxCourant;
}
```

```
// Utilisation :
List<Integer> nombres = List.of(3, 9, 2, 7);
System.out.println(max(nombres, Comparator.naturalOrder())); // 9
```

Exercice 7 — Correction

```
public class Boite<T> {
    private T contenu;
    public void setContenu(T contenu) { this.contenu = contenu; }
    public T getContenu() { return contenu; }
}

Boite<String> b1 = new Boite<>();
b1.setContenu("Bonjour");

Boite<Integer> b2 = new Boite<>();
b2.setContenu(42);
```

Exercice 8 — Correction

```
List<Personne> liste = new ArrayList<>();
// ... ajout de personnes ...

liste.sort(
    Comparator.comparing(Personne::getNom)
                .thenComparing(Personne::getAge)
);
```

Exercice 9 — Correction

```
Map<String, Integer> hash = new HashMap<>();
Map<String, Integer> linked = new LinkedHashMap<>();

for (Map<String, Integer> m : List.of(hash, linked)) {
    m.put("Tom", 17);
    m.put("Ainhua", 21);
    m.put("Marie", 30);
}

System.out.println(hash); // ordre non garanti
System.out.println(linked); // ordre d'insertion conservé : {Tom=17, Ainhua=21, Marie=30}
```

Exercice 10 — Correction

```
Map<String, List<Personne>> groupes = new HashMap<>();
groupes.put("mineur", new ArrayList<>());
groupes.put("majeur", new ArrayList<>());

for (Personne p : personnes) {
    if (p.getAge() >= 18) {
        groupes.get("majeur").add(p);
    } else {
        groupes.get("mineur").add(p);
    }
}
```

Thème 4 — Streams & lambdas

Exercice 1 — Correction

```
List<Integer> nombres = new ArrayList<>(List.of(3, 9, 2, 7));
nombres.sort((a, b) -> b - a);
System.out.println(nombres); // [9, 7, 3, 2]
```

Exercice 2 — Correction

```
interface Operation {
    int appliquer(int a, int b);
}

Operation soustraction = (a, b) -> a - b;
System.out.println(soustraction.appliquer(10, 4)); // 6
```

Exercice 3 — Correction

```
List<String> mots = List.of("Ainhua", "Marie", "Elephant", "Tom");

List<String> resultat = mots.stream()
    .filter(m -> "AEIOUaeiou".indexOf(m.charAt(0)) >= 0)
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(resultat); // [AINHUA, ELEPHANT]
```

Exercice 4 — Correction

```
int sommeAges = personnes.stream()
    .mapToInt(Personne::getAge)
    .sum();

System.out.println(sommeAges);
```

Exercice 5 — Correction

```
List<String> mots = List.of("Java", "est", "cool");

String resultat = mots.stream()
    .reduce((a, b) -> a + "-" + b)
    .orElse("");

System.out.println(resultat); // Java-est-cool
```

Exercice 6 — Correction

```
long nbMajeurs = personnes.stream()
    .filter(p -> p.getAge() >= 18)
    .count();

System.out.println(nbMajeurs);
```

Exercice 7 — Correction

```
List<Personne> trieEs = personnes.stream()
    .sorted(Comparator.comparing(Personne::getAge).reversed())
    .collect(Collectors.toList());
```

Exercice 8 — Correction

```
List<String> mots = List.of("chat", "chien", "rat", "souris");

Map<Integer, List<String>> parLongueur = mots.stream()
    .collect(Collectors.groupingBy(String::length));

System.out.println(parLongueur); // {3=[rat], 4=[chat], 5=[chien], 6=[souris]}
```

Exercice 9 — Correction

```
double moyenneAge = personnes.stream()
    .filter(p -> p.getAge() >= 18)
    .mapToInt(Personne::getAge)
    .average()
    .orElse(0);

System.out.println(moyenneAge);
```

Exercice 10 — Correction

```
public static List<Personne> filtrerPersonnes(List<Personne> liste, Predicate<Personne> condition) {
    return liste.stream()
        .filter(condition)
        .collect(Collectors.toList());
}

// Utilisation :
List<Personne> majeurs = filtrerPersonnes(personnes, p -> p.getAge() >= 18);
```


Thème 5 — Design Patterns

Exercice 1 — Correction

```
public class Logger {
    private static Logger instance;

    private Logger() {}

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}
```

Exercice 2 — Correction

```
public interface Vehicule { void rouler(); }

public class Voiture implements Vehicule {
    public void rouler() { System.out.println("La voiture roule"); }
}
public class Moto implements Vehicule {
    public void rouler() { System.out.println("La moto roule"); }
}

public class VehiculeFactory {
    public static Vehicule creer(String type) {
        return switch (type) {
            case "voiture" -> new Voiture();
            case "moto" -> new Moto();
            default -> throw new IllegalArgumentException("Type inconnu");
        };
    }
}
```

Exercice 3 — Correction

```
public class Burger {
    private final String pain;
    private final String viande;
    private final boolean fromage;

    public static class Builder {
        private String pain = "classique";
        private String viande = "boeuf";
        private boolean fromage = false;

        public Builder pain(String p) { this.pain = p; return this; }
        public Builder viande(String v) { this.viande = v; return this; }
        public Builder fromage(boolean f) { this.fromage = f; return this; }
        public Burger build() { return new Burger(this); }
    }

    private Burger(Builder b) {
        this.pain = b.pain;
        this.viande = b.viande;
        this.fromage = b.fromage;
    }
}

Burger b = new Burger.Builder().viande("poulet").fromage(true).build();
```

Exercice 4 — Correction

```

public interface Afficheur { void miseAJour(double temperature); }

public class MeteoStation {
    private List<Afficheur> afficheurs = new ArrayList<>();

    public void abonner(Afficheur a) { afficheurs.add(a); }

    public void changerTemperature(double t) {
        for (Afficheur a : afficheurs) {
            a.miseAJour(t);
        }
    }
}

public class AfficheurConsole implements Afficheur {
    public void miseAJour(double t) {
        System.out.println("Nouvelle température : " + t);
    }
}

```

Exercice 5 — Correction

```

public interface StrategieTri { void trier(List<Integer> liste); }

public class TrierCroissant implements StrategieTri {
    public void trier(List<Integer> liste) { liste.sort(Comparator.naturalOrder()); }
}

public class TrierDecroissant implements StrategieTri {
    public void trier(List<Integer> liste) { liste.sort(Comparator.reverseOrder()); }
}

public class Trieur {
    private StrategieTri strategie;
    public Trieur(StrategieTri s) { this.strategie = s; }
    public void executer(List<Integer> liste) { strategie.trier(liste); }
}

```

Exercice 6 — Correction

```

public enum Logger {
    INSTANCE;

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}

// Utilisation :
Logger.INSTANCE.log("Démarrage de l'application");
// La JVM garantit la création d'une seule instance, de façon thread-safe.

```

Exercice 7 — Correction

```

public interface StrategiePaiement { void payer(double montant); }

public class PaiementCarte implements StrategiePaiement {
    public void payer(double m) { System.out.println("Carte : " + m); }
}

public class PaiementEspèces implements StrategiePaiement {
    public void payer(double m) { System.out.println("Espèces : " + m); }
}

public class PaiementFactory {
    public static StrategiePaiement creer(String type) {
        return switch (type) {

```

```
        case "carte" -> new PaiementCarte();
        case "especes" -> new PaiementEspeces();
        default -> throw new IllegalArgumentException("Type inconnu");
    };
}
}
```

Exercice 8 — Correction

```
MeteoStation station = new MeteoStation();
station.abonner(new AfficheurConsole());
station.abonner(new AfficheurConsole()); // un deuxième abonné

station.changerTemperature(24.5);
// Les deux afficheurs affichent le message
```

Exercice 9 — Correction

```
Pizza p1 = new Pizza.Builder().taille("L").fromage(true).build();
Pizza p2 = new Pizza.Builder().fromage(true).build(); // taille par défaut "M"
```

Exercice 10 — Correction

```
public class ConnexionBD {
    private static ConnexionBD instance;
    private ConnexionBD() {
        // initialisation de la connexion réelle ici
    }
    public static ConnexionBD getInstance() {
        if (instance == null) {
            instance = new ConnexionBD();
        }
        return instance;
    }
}
// Pattern Singleton : garantit une seule instance de connexion partagée.
```

Thème 6 — JUnit 5 & JDBC

Exercice 1 — Correction

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculTest {
    @Test
    void testDivision() {
        Calcul calcul = new Calcul();
        assertEquals(5, calcul.diviser(10, 2));
    }
}
```

Exercice 2 — Correction

```
@Test
void testDivisionParZero() {
    Calcul calcul = new Calcul();
    assertThrows(ArithmeticException.class, () -> calcul.diviser(10, 0));
}
```

Exercice 3 — Correction

```
class CalculTest {
    private Calcul calcul;

    @BeforeEach
    void setup() {
        calcul = new Calcul();
    }

    @Test
    void testAddition() {
        assertEquals(7, calcul.additionner(3, 4));
    }
}
```

Exercice 4 — Correction

```
@Test
void testEstPositifValeur() {
    assertEquals(true, Utils.estPositif(5));
}

@Test
void testEstPositifTrue() {
    assertTrue(Utils.estPositif(10));
}

@Test
void testEstPositifFalse() {
    assertFalse(Utils.estPositif(-3));
}
```

Exercice 5 — Correction

```
class ExempleTest {
    @BeforeAll
    static void init() {
        System.out.println("Début des tests");
    }

    @Test
    void test1() { /* ... */ }
}
```

Exercice 6 — Correction

```
String url = "jdbc:mysql://localhost:3306/madb";
try (Connection conn = DriverManager.getConnection(url, "root", "motdepasse")) {
    String sql = "SELECT nom, age FROM personnes WHERE age > ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, 18);
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                System.out.println(rs.getString("nom") + " - " + rs.getInt("age"));
            }
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

Exercice 7 — Correction

```
String sql = "INSERT INTO personnes (nom, age) VALUES (?, ?)";
try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, "Ainhua");
    stmt.setInt(2, 21);
    stmt.executeUpdate();
}
```

Exercice 8 — Correction

```
// Avec Statement : vulnérable à l'injection SQL
String sql = "SELECT * FROM personnes WHERE nom = '" + nomSaisi + "'";

// Avec PreparedStatement : sûr, le paramètre est échappé automatiquement
String sqlSecurise = "SELECT * FROM personnes WHERE nom = ?";
PreparedStatement stmt = conn.prepareStatement(sqlSecurise);
stmt.setString(1, nomSaisi);
```

Exercice 9 — Correction

```
public Personne getPersonneParNom(String nom) throws SQLException {
    String sql = "SELECT nom, age FROM personnes WHERE nom = ?";
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, nom);
        try (ResultSet rs = stmt.executeQuery()) {
            if (rs.next()) {
                return new Personne(rs.getString("nom"), rs.getInt("age"));
            }
        }
    }
    return null;
}
```

Exercice 10 — Correction

```
// Avec Mockito, on simule (mock) la Connection/PreparedStatement/ResultSet
// pour tester la logique du DAO sans base de données réelle :
```

```
@Test
void testGetPersonneParNom() throws SQLException {
    Connection connMock = mock(Connection.class);
    PreparedStatement stmtMock = mock(PreparedStatement.class);
    ResultSet rsMock = mock(ResultSet.class);

    when(connMock.prepareStatement(anyString())).thenReturn(stmtMock);
    when(stmtMock.executeQuery()).thenReturn(rsMock);
    when(rsMock.next()).thenReturn(true);
    when(rsMock.getString("nom")).thenReturn("Ainhua");
    when(rsMock.getInt("age")).thenReturn(21);

    PersonneDAO dao = new PersonneDAO(connMock);
```

```
    Personne p = dao.getPersonneParNom("Ainhua");  
    assertEquals("Ainhua", p.getNom());  
}
```